University of Engineering & Management, Kolkata
Institute of Engineering and Management, Newtown Campus
Department of Computer Applications

ASSIGNMENT SUBMISSION — TOP SHEET

Technical Class Assessment — April 2026
Database Management Systems & Java with JDBC | MCA Programme

STUDENT DETAILS

Full Name	Sourav Mukherjee		
Roll Number	38	**Registration No.**	304202500600098
Section	B	**Semester**	2
Programme	MCA	**Academic Year**	2025-2026
Email ID	sourav.mukherjee2025@iem.edu.in		

ASSIGNMENT DETAILS

Subject	Java - DBMS Assignment	**Set Number**	19
Problem Title	The Blood Pressure Diary		
Language Used	DBMS & Core Java	**Database Used**	MySQL 8.0.44
Submission Date	10.04.26	**Total Pages**	12

SUBMISSION CHECKLIST

#	Item	Included (✓ / ✗)	Pages / Remarks
1	Printed Java Source Code (DBConnection.java)		
2	Printed Java Source Code (Main.java)		
3	Printed SQL Schema File (schema.sql)		
4	Screenshot(s) of Program Execution (printed)		
5	Problem Statement (this top sheet attached)		

DECLARATION

I hereby declare that this assignment is my own original work. I have not copied from any other student or used any unauthorised sources. I understand that plagiarism will result in zero marks and disciplinary action.

Student Signature & Date

Faculty Signature & Date

University of Engineering & Management, Kolkata
Institute of Engineering and Management, Newtown Campus
Department of Computer Applications

TECHNICAL CLASS ASSESSMENT — April – 2026

Degree: MCA	Stream: MCA
Due Date: _____	**Set No: 19**

🩺 The Blood Pressure Diary

[Domain: Personal Health Monitoring]

Language	Core Java (Console Application)
Database	MySQL
Connectivity	JDBC (Java Database Connectivity)

The Story

> *"She had seven high readings in a row. We only found out after she was hospitalised."*

A community health worker in a Kolkata slum visits elderly residents and records their blood pressure readings in a notebook. Trends are never analysed, dangerous readings are not flagged, and doctors are informed only when a crisis occurs.

Your task is to build a Blood Pressure Monitoring System using Java and MySQL.

Your Task

Build a Blood Pressure Monitoring System as a Java console application using JDBC and MySQL.

Your system must allow:

- Register a patient (name, age, address, contact)
- Record a BP reading (systolic, diastolic, date and time) for a patient
- Flag reading as high if systolic > 140 or diastolic > 90
- Display last 5 readings for a patient and count of high readings

Database Schema

Create the following table(s) in MySQL:

```
CREATE TABLE patients (
    pat_id   INT PRIMARY KEY AUTO_INCREMENT,
    name     VARCHAR(100),
    age      INT,
    address  VARCHAR(200),
    contact  VARCHAR(15)
);
```

```
CREATE TABLE readings (
    read_id   INT PRIMARY KEY AUTO_INCREMENT,
    pat_id    INT,
    systolic INT,
    diastolic INT,
    is_high   BOOLEAN,
    read_time DATETIME DEFAULT NOW(),
    FOREIGN KEY (pat_id) REFERENCES patients(pat_id)
);
```

Required Features

Feature	Implementation Hint
Register Patient	INSERT into patients with name, age, address, contact
Record Reading	INSERT; set is_high = (systolic > 140 OR diastolic > 90)
Last 5 Readings	SELECT * FROM readings WHERE pat_id=? ORDER BY read_time DESC LIMIT 5
High Count	SELECT COUNT(*) WHERE pat_id=? AND is_high=TRUE

Constraints

- Use Core Java only — no Spring, Hibernate, or web frameworks
- Connect to MySQL using JDBC (no JPA / ORM)
- All operations must be menu-driven via console
- No GUI required — Scanner-based input is sufficient
- Handle SQL exceptions with try-catch blocks
- Use PreparedStatement for all SQL queries

Sample Interaction

```
===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 2
Patient ID: 3  Systolic: 155  Diastolic: 95
Reading recorded. Status: HIGH BP [ALERT]
```

Submission Checklist

- DBConnection.java (JDBC connection class)
- Main.java (menu-driven main program)
- SQL file: schema.sql (CREATE TABLE statements)
- screenshot.png (terminal output showing at least 2 features working)

> *"A database is not just data. It is the memory of your application. Make it reliable."*

src/Main.java

```
import java.sql.*;
import java.util.Scanner;

public class Main {

    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        while (true) {
            System.out.println("\n===== BP Monitor System =====");
            System.out.println("1. Register Patient");
            System.out.println("2. Record Reading");
            System.out.println("3. View Last 5 Readings");
            System.out.println("4. Count High Readings");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");

            int choice = sc.nextInt();

            switch (choice) {
                case 1:
                    registerPatient();
                    break;
                case 2:
                    recordReading();
                    break;
                case 3:
                    viewLastReadings();
                    break;
                case 4:
                    countHighReadings();
                    break;
                case 5:
                    System.exit(0);
                default:
                    System.out.println("Invalid choice!");
            }
        }
    }

    // 1. Register Patient
```

```
public static void registerPatient() {
    try (Connection conn = DBConnection.getConnection()) {

        System.out.print("Name: ");
        sc.nextLine(); // consume leftover
        String name = sc.nextLine();

        System.out.print("Age: ");
        int age = sc.nextInt();

        sc.nextLine();
        System.out.print("Address: ");
        String address = sc.nextLine();

        System.out.print("Contact: ");
        String contact = sc.nextLine();

        String query = "INSERT INTO patients (name, age, address, contact) VALUES (?, ?, ?, ?)";

        PreparedStatement ps = conn.prepareStatement(query);
        ps.setString(1, name);
        ps.setInt(2, age);
        ps.setString(3, address);
        ps.setString(4, contact);

        ps.executeUpdate();

        System.out.println("Patient registered successfully!");

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// 2. Record Reading
public static void recordReading() {
    try (Connection conn = DBConnection.getConnection()) {

        System.out.print("Patient ID: ");
        int pat_id = sc.nextInt();

        System.out.print("Systolic:  ");
```

```
        int sys = sc.nextInt();

        System.out.print("Diastolic: ");
        int dia = sc.nextInt();

        boolean isHigh = (sys > 140 || dia > 90);

        String query = "INSERT INTO readings (pat_id, systolic, diastolic, is_high) VALUES (?, ?, ?, ?)";

        PreparedStatement ps = conn.prepareStatement(query);
        ps.setInt(1, pat_id);
        ps.setInt(2, sys);
        ps.setInt(3, dia);
        ps.setBoolean(4, isHigh);

        ps.executeUpdate();

        if (isHigh) {
            System.out.println("Reading recorded. Status: HIGH BP [ALERT]");
        } else {
            System.out.println("Reading recorded. Status: NORMAL");
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// 3. View Last 5 Readings
public static void viewLastReadings() {
    try (Connection conn = DBConnection.getConnection()) {

        System.out.print("Patient ID: ");
        int pat_id = sc.nextInt();

        String query = "SELECT * FROM readings WHERE pat_id=? ORDER BY read_time DESC LIMIT 5";

        PreparedStatement ps = conn.prepareStatement(query);
        ps.setInt(1, pat_id);

        ResultSet rs = ps.executeQuery();
```

```
            System.out.println("\nLast 5 Readings:");

            while (rs.next()) {
                System.out.println(
                        "Sys: " + rs.getInt("systolic") +
                                " Dia: " + rs.getInt("diastolic") +
                                " High: " + rs.getBoolean("is_high") +
                                " Time: " + rs.getTimestamp("read_time")
                );
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }

    // 4. Count High Readings
    public static void countHighReadings() {
        try (Connection conn = DBConnection.getConnection()) {

            System.out.print("Patient ID: ");
            int pat_id = sc.nextInt();

            String query = "SELECT COUNT(*) FROM readings WHERE pat_id=? AND is_high=TRUE";

            PreparedStatement ps = conn.prepareStatement(query);
            ps.setInt(1, pat_id);

            ResultSet rs = ps.executeQuery();

            if (rs.next()) {
                System.out.println("High BP Count: " + rs.getInt(1));
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

src/DBConnection.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/bp_monitor";
    private static final String USER = "root";      // change if needed
    private static final String PASSWORD = "123456"; // change

    public static Connection getConnection() {
        try {
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (SQLException e) {
            System.out.println("Database Connection Failed!");
            e.printStackTrace();
            return null;
        }
    }
}
```

sql/schema.sql

```sql
CREATE DATABASE bp_monitor;
USE bp_monitor;

CREATE TABLE patients (
   pat_id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100),
   age INT,
   address VARCHAR(200),
   contact VARCHAR(15)
);

CREATE TABLE readings (
   read_id INT PRIMARY KEY AUTO_INCREMENT,
   pat_id INT,
   systolic INT,
   diastolic INT,
   is_high BOOLEAN,
   read_time DATETIME DEFAULT NOW(),
   FOREIGN KEY (pat_id) REFERENCES patients(pat_id)
);
```

Screenshots:

```
===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 1
Name: Soumya Biswas
Age: 21
Address: Barrackpore
Contact: 8617714895
Patient registered successfully!

===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 2
Patient ID: 1
Systolic: 140
Diastolic: 90
Reading recorded. Status: NORMAL
```

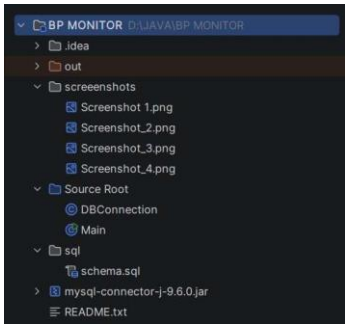
```
===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 1
Name: Soumya Sharma
Age: 22
Address: Kharagpur
Contact: 8975642354
Patient registered successfully!

===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 2
Patient ID: 2
Systolic: 145
Diastolic: 95
Reading recorded. Status: HIGH BP [ALERT]
```

```
===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 3
Patient ID: 1

Last 5 Readings:
Sys: 140 Dia: 90 High: false Time: 2026-04-10 00:19:54.0
Sys: 140 Dia: 90 High: false Time: 2026-04-09 22:55:53.0

===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 4
Patient ID: 2
High BP Count: 1
```

```
===== BP Monitor System =====
1. Register Patient
2. Record Reading
3. View Last 5 Readings
4. Count High Readings
5. Exit
Enter choice: 5
```

Folder Structure: